phone_app/lib/phone_app_web/controllers/message_html.ex
```
attr :type, :string, default: "button", values: ["button", "submit"]
attr :text, :string, required: true

def simple_button(assigns) do
  ~H"""
  <button
    type={@type}
    class="rounded border bg-white text-gray-700 px-4 py-2"
  >
    <%= @text %>
  </button>
  """
end
```

Everything inside of the ~H sigil will be rendered when we use the simple_component:

```
<.simple_button text="Deliver" type="submit" />
```

If we wanted to have HTML inside of our button, it wouldn't work with this implementation. We can use slots to create a better button interface:

phone_app/lib/phone_app_web/controllers/message_html.ex
```
attr :type, :string, default: "button", values: ["button", "submit"]
slot :inner_block, required: true

def slot_button(assigns) do
  ~H"""
  <button
    type={@type}
    class="rounded border bg-white text-gray-700 px-4 py-2"
  >
    <%= render_slot(@inner_block) %>
  </button>
  """
end
```

Now we can pass in HTML for the button contents:

```
<.slot_button type="submit">
  <strong>Strong Button!</strong>
</.slot_button>
```

This example uses the default slot called inner_block, but you can have as many slots as you want by naming the slot. Named slots let you create a component that consumes and renders multiple content areas. Slots can do even more, so make sure to check out the excellent docs.[25]

25. https://hexdocs.pm/phoenix_live_view/Phoenix.Component.html#module-slots